High-Level Design (HLD)

RAG-Based Customer Support Assistant

1. System Overview

Problem Definition

Traditional customer support systems fail to provide accurate answers from large documents. They either return generic responses or require manual searching.

Proposed Solution

This system uses **Retrieval-Augmented Generation (RAG)** to:

- Retrieve relevant information from a PDF knowledge base
- Generate context-aware answers using an LLM

Scope

- Input: User query
- Output: Accurate answer based on PDF
- Domain: Customer support / knowledge retrieval

2. Architecture Overview

The system consists of the following components:

- User Interface (CLI)
- Document Ingestion Pipeline
- Text Chunking Module
- Embedding System
- Vector Database (ChromaDB)
- Retrieval Layer
- LLM Processing Layer (Groq)
- Workflow Orchestration (LangGraph)
- Human-in-the-Loop (HITL) System

3. Architecture Flow

1. User enters query
2. Query is processed
3. Relevant document chunks are retrieved
4. Context is passed to LLM
5. LLM generates answer
6. If confidence is low → escalate to human
7. Final answer is displayed

4. Component Description

Document Loader

Loads PDF file using PyPDFLoader

Chunking Strategy

Splits text into smaller chunks (approx. 800 characters)
Ensures better retrieval accuracy

Embedding Model

Converts text into vector form using HuggingFace model

Vector Store

Stores embeddings in ChromaDB for fast similarity search

Retriever

Fetches top-k relevant chunks based on query

LLM

Generates final answer using Groq model

Graph Workflow Engine

Controls execution flow using LangGraph

Routing Layer

Decides whether to:

- Send to output
- Escalate to human

HITL Module

Handles human intervention for low-confidence queries

5. Data Flow

1. PDF → Loaded
2. Text → Chunked
3. Chunks → Converted to embeddings
4. Stored in ChromaDB
5. User query → Embedded
6. Similar chunks retrieved
7. Context passed to LLM
8. Answer generated

6. Technology Choices

ChromaDB

- Lightweight and easy to use
- Efficient vector storage

LangGraph

- Controls workflow logic
- Enables conditional routing

Groq LLM

- Fast inference
- High-quality responses

LangChain

- Integrates all components

7. Scalability Considerations

- Handles large documents using chunking
- Supports multiple queries efficiently
- Uses vector search for fast retrieval
- Reduces latency using Groq

Conclusion

This system provides an efficient and scalable solution for customer support using RAG. It improves accuracy, reduces manual effort, and enables intelligent query handling with workflow-based control.